Dataset and Exploratory Data Analysis Report

Project Title: Hybrid GNN-Based NOMA for User Assignment and Slot Pairing

Prepared By:

Monica Raghini Chelle

Simulation Reconstruction & Statistical Profiling

October 14, 2026

Contents

Executive Summary 1

1 Dataset Source and Generation 1

2 Sample Units and Split Design 1

3 Variables and Labels 3

4 Descriptive Statistics 3

5 Figures 5

6 Data Quality and Outlier Screening 5

7 Interpretation and Limitations 5

8 Files and How to Reproduce 6

Executive Summary

The implemented GNN is trained on **synthetic wireless-network graphs**, not measured or externally sourced radio data. The configured default is 200 training scenarios, 40 validation scenarios, and 50 separately generated benchmark scenarios; each contains 10 users and has oracle-derived pair and slot labels. This package contains **290 scenarios, 2,900 user rows, and 26,100 directed non-self edges**.

The raw historical scenarios are not saved by the project. Consequently, these files are a documented, reproducible **reconstruction** of the configured simulation and labels, not a claim to recover byte-for-byte NumPy samples from an earlier run. The reconstruction uses Python's standard-library random generator. See `README.md` and `dataset_metadata.json` before interpreting or citing the numeric results.

1 Dataset Source and Generation

The source project constructs users in memory in `src/channel_model.py`: users are uniform over an annulus with radii 20–500 m; path loss follows the configured log-distance law; independent complex Gaussian fading produces Rayleigh small-scale channel magnitudes. The GNN's dataset generator in `src/training.py` creates a graph per scenario and obtains labels from the exhaustive oracle in `src/optimization.py`. The graph builder in `src/graph_builder.py` forms a complete directed graph without self-loops and derives six node and four edge input features.

There are no real-world measurement records or imported external training datasets in this primary GNN path. `data/example_users.csv` in the source project is a small illustrative input and is not the default GNN training set. The separate optional classical-regression workflow creates rate/outage targets from simulator formulas; those are included here only as auxiliary reference columns, not as GNN labels.

Provenance and Reproducibility Qualification

In the project, train and validation seeds are set by `cfg.system.seed + seed_offset + index`; `main.py` uses offsets 100 and 500, respectively. Evaluation uses seed `seed_offset + index` with offset 1000. This reconstruction follows those seed numbers and split sizes, along with the simulator equations, graph features, and exhaustive pairing/scheduling objective. It uses Python's `random.Random`, whereas the project uses NumPy `default_rng`; identical seed integers therefore **do not** produce the same random samples. No saved raw scenario cache was found to compare against. To export the exact samples of a future project run, modify/use a project-side exporter when its dependencies are available; do not treat these reconstructed draws as archived run data.

2 Sample Units and Split Design

The independent observational unit is the **scenario/graph**, not the individual user row. Users within a scenario share a topology and are statistically dependent. The scenario seeds are disjoint across the three sets.

Training and validation are the GNN's supervised-learning splits. The 50 benchmark scenarios are a separately generated simulator benchmark set; they are not a classical-regression train/validation/test partition. The configuration has five slots and ten users per graph, so the exhaustive oracle assigns five pairs per graph and one pair to each slot.

Set	Scenarios (graphs)	User rows	Seed range
train	200	2,000	142–341
validation	40	400	542–581
benchmark	50	500	1000–1049

Table 1: Dataset Split Summary

Split-Level Distribution Check

The values below are mean / population SD across user rows in each split. These are a descriptive drift check; the scenario remains the independent unit.

Split	Distance (m), mean / SD	Channel gain (dB), mean / SD	Per-slot SNR (dB), mean / SD
train	337.85 / 115.85	-127.98 / 9.2122	1.0166 / 9.2122
validation	339.5 / 119.73	-128.16 / 9.5128	0.84079 / 9.5128
benchmark	334.43 / 114.04	-127.78 / 9.507	1.224 / 9.507

Table 2: Split-level User Feature Metrics

Oracle outcome summaries are scenario-level mean / SD:

Split	Sum rate (Mbps), mean / SD	Jain fairness, mean / SD	Outage users/scenario, mean / SD
train	20.217 / 7.316	0.52647 / 0.10986	0.52 / 0.84829
validation	20.165 / 5.7709	0.50282 / 0.10032	0.6 / 0.88882
benchmark	21.153 / 7.464	0.5269 / 0.11643	0.58 / 0.89644

Table 3: Oracle Outcome Summaries

3 Variables and Labels

Physical/User-Level Fields

Each split's `*_users.csv` has one row per user and retains split/scenario/seed identifiers, coordinates, BS distance, path-loss values, complex channel components, linear and dB channel gain, and per-slot SNR. `rate_target_bps` and `outage_target` are auxiliary, simulator-derived single-user targets following `src/data_pipeline.py`; they are not GNN supervision.

GNN Inputs

Node features: `node_x_norm`, `node_y_norm`, `node_distance_norm`, `node_gain_db_zscore`, `node_gain_relativ` `node_min_rate_mbps`.

Directed-edge features: `edge_distance_norm`, `edge_gain_disparity_norm`, `edge_channel_correlation` `edge_interference_potential`. For 10 users, each graph has $10 \times 9 = 90$ directed non-self edges.

The configured YAML comment calls the sixth node feature "target_rate", but the implementation uses `min_rate_req_bps / 1e6`. Here it is 0.1 Mbps for every user, so that feature is constant and does not convey a user-specific achieved/target throughput.

Supervision

The oracle supplies a symmetric pair adjacency target and one slot ID per user. In the exported user rows, `oracle_partner_user_id` and `oracle_slot_id` encode those targets; `oracle_mode` and `oracle_user_rate_bps` record the oracle's selected link mode and achieved per-user rate. The edge file contains the pair target as `is_oracle_pair`. Labels are generated from the same simulator and optimization objective (sum rate plus weighted Jain fairness); they are optimization targets, not ground-truth radio measurements.

Slot label	User rows
0	580
1	580
2	580
3	580
4	580

Oracle selected pair mode	User rows
PD-NOMA	2,598
SCMA	302

Pair-label balance: **2,900 positive** and **23,200 negative** directed edges (11.11% positive). Each fully paired graph contributes 10 positive directed edges and 80 negatives. Any training interpretation should account for this imbalance (the project uses a positive-class weight in its BCE loss).

4 Descriptive Statistics

Statistics below pool user rows across train, validation, and benchmark sets. Standard deviation is population SD. User-row summaries are descriptive; they do not make 2,900 independent scenarios.

Feature Relationships

Pearson correlations use pooled user rows. Correlation is descriptive and does not imply causal effects; the physical generation process and within-scenario clustering matter.

Variable	N	Mean	Pop. SD	Min	Max
x_m	2,900	4.499	249.496	-498.909	497.792
y_m	2,900	1.702	255.153	-497.459	497.794
distance_m	2,900	337.485	116.097	21.268	499.676
channel_gain_linear	2,900	1.386×10^{-11}	1.935×10^{-10}	2.628×10^{-18}	5.407×10^{-9}
channel_gain_db	2,900	-127.972	9.306	-175.804	-82.671
snr_db	2,900	1.028	9.306	-46.804	46.329
min_rate_req_bps	2,900	100,000	0	100,000	100,000

Table 4: Pooled User Row Descriptive Statistics

Feature	x_m	y_m	distance_m	gain_db	snr_db
x_m	1.000	0.021	0.014	0.009	0.009
y_m	0.021	1.000	0.005	-0.008	-0.008
distance_m	0.014	0.005	1.000	-0.752	-0.752
channel_gain_db	0.009	-0.008	-0.752	1.000	1.000
snr_db	0.009	-0.008	-0.752	1.000	1.000

Table 5: Pearson Correlation Matrix

The edge feature named `edge_channel_correlation` is computed in the source as the magnitude of a normalized product of two **scalar** complex channel coefficients. For single-antenna scalar channels, this is 1 whenever both magnitudes exceed the numerical threshold (otherwise 0); it is not a spatial/antenna correlation model. It is 1 for every exported edge in this reconstruction, so it is a constant feature here. This deserves explicit mention in project interpretation and could be reconsidered in a future code revision; no source-code change is made here.

The `node_min_rate_mbps` feature is constant at 0.1 for the default configuration. Its correlation with nonconstant variables is undefined, so it is not included in the correlation matrix.

5 Figures

The visual artifacts generated by the analysis script are referenced below:

- `figures/distance_distribution.svg` — User distance distribution
- `figures/channel_gain_distribution.svg` — Channel gain distribution
- `figures/snr_distribution.svg` — Per-slot SNR distribution
- `figures/training_user_locations.svg` — Training user locations
- `figures/feature_correlation.svg` — Feature correlation heatmap

Location plot colors encode channel gain dB; the center marker is the base station. Histograms describe per-user values, not scenario-level outcomes.

6 Data Quality and Outlier Screening

Automated checks on the exported data found **2,900 user rows**, **290 scenarios**, **0 duplicate scenario/user keys**, no missing or non-finite values among checked numeric inputs/features/targets, distances within the configured annulus, reciprocal pair assignments, and all slot labels within 0–4. The largest absolute difference between $\sqrt{x^2 + y^2}$ and stored distance is 5.68×10^{-14} m (floating-point rounding).

An IQR 1.5×fence screen flags rows for review; it does not label observations as erroneous. In this simulator, large/small channel gains and SNRs can naturally occur under Rayleigh fading, and distance endpoints follow the configured annulus.

Variable	Rows outside IQR fences	Lower fence	Upper fence
`x_m`	0	-801.51	807.013
`y_m`	0	-844.863	847.024
`distance_m`	0	-7.944	700.651
`channel_gain_linear`	420	-5.691×10^{-13}	1.068×10^{-12}
`channel_gain_db`	134	-148.607	-108.326
`snr_db`	134	-19.607	20.674
`min_rate_req_bps`	0	100,000	100,000

Table 6: IQR Outlier Screening Summary

No flagged observation was automatically deleted or winsorized.

7 Interpretation and Limitations

1. **Synthetic scope:** EDA describes the configured simulation, not deployment conditions, measured users, or a field trial. Do not present it as empirical radio data.
2. **Reconstruction, not historical recovery:** the project stores graph/metric outputs rather than raw generated train/validation/benchmark channel rows. This package is reproducible but uses a different RNG implementation from the project.
3. **Oracle dependence:** pair and slot labels are optimal under this simulator's finite-slot objective and assumptions. They are not independent annotations and may not transfer to another propagation, interference, or traffic model.

4. **Scenario clustering:** inferential statistics must resample or split by scenario, not by user/edge row.
5. **Rate/outage columns:** these are calculated from a single-user Shannon-rate proxy over one slot. They are not the selected NOMA pair rates and are not GNN targets.
6. **SCMA representation:** the hybrid schedule's pair-mode choice uses a simplified rate proxy in `src/slot_assignment.py`. It should not be interpreted as end-to-end SCMA message-passing detector evidence. The project's SCMA BER evaluation is a different experiment.
7. **Model feature semantics:** minimum-rate requirement is constant in the default setup; scalar channel "correlation" is nearly constant. Such features provide little information under this configuration.
8. **Hardware parameters:** 2.4 GHz is configured, but the current path-loss implementation uses a fixed reference loss and exponent; carrier frequency is not separately applied in the per-user gain formula.

8 Files and How to Reproduce

- `train_users.csv`, `validation_users.csv`, `benchmark_users.csv`: split-specific user state, graph input features, and oracle labels.
- `all_splits_edges.csv`: directed non-self edge rows, edge inputs, and pair targets.
- `scenario_summary.csv`: one row per graph with oracle outcomes.
- `feature_correlations.csv`: correlation matrix values.
- `dataset_metadata.json`: full configuration, schema, computed statistics, labels, and quality checks.
- `figures/`: vector EDA plots.
- `generate_dataset_eda.py`: standard-library-only generator and report builder.

Run from any directory with Python 3.10 or later:

```
python Dataset\generate_dataset_eda.py
```

The script overwrites only generated files in this `Dataset` directory. For academically accurate reporting, cite the project implementation/configuration and explicitly describe these files as a simulator reconstruction, unless the historical raw samples are later exported from the exact project run.